

Knowledge Graphs for AI Systems (COMP6256)

Coursework

Querying and Retrieving Structured Facts from Wikidata using SPARQL

- Harsh Kumar

This coursework explores how structured facts can be retrieved from a large knowledge graph using SPARQL queries. The tasks were completed using the Wikidata Query Service (WDQS), which provides access to the Wikidata knowledge graph through the SPARQL query language. Each task required translating a natural language question into a structured graph query that navigates entities, properties, and relationships within the dataset.

While implementing the queries, I focused on carefully interpreting the modelling structure used in Wikidata rather than assuming a traditional relational database design. In several cases, the tasks required traversing multi-step relationships, aggregating results across connected entities, or identifying missing information in the graph. As a result, the queries make use of a range of SPARQL features including grouping and aggregation, property paths, negation patterns, and missing-data detection.

For each task, I provide a short explanation of the reasoning behind the query structure and include the exact SPARQL code used in the corresponding *.rq* file, as required in the coursework brief. The outputs shown in this report were obtained directly from the Wikidata Query Service. In situations where the public endpoint presented modelling or performance limitations, I verified the correctness of the query logic using smaller preview queries and documented those constraints transparently.

Overall, the goal of this report is not only to present working queries but also to demonstrate an understanding of how structured information is represented and retrieved within a real-world knowledge graph.

Task 1: The “Lonely” Countries

Task: Count sovereign states that share a land border with exactly one other country.

For this task, I needed to identify sovereign states and then examine how many distinct neighbouring countries each one borders. Since the question asks for countries with exactly one neighbour, I approached it as an aggregation problem. I grouped the results by country, counted the distinct border neighbours for each one, and then filtered the grouped results so that only countries with exactly one neighbour remained. After that, I counted how many such countries existed in total.

SPARQL Query:

Task 1: The "Lonely" Countries

Objective: Count the number of sovereign states that share a land border with exactly one other country.

The outer query counts how many countries satisfy the "lonely country" condition.

SELECT (COUNT(*) AS ?lonelyCountryCount)

WHERE {

Subquery used to first identify which countries have exactly one border neighbour.

{

SELECT ?country

WHERE {

Restrict the entities to sovereign states.

P31 = instance of

Q3624078 = sovereign state

?country wdt:P31 wd:Q3624078 .

Identify neighbouring countries using the property:

P47 = shares border with

?country wdt:P47 ?neighbour .

}

Group results by each country so that we can count its neighbours.

GROUP BY ?country

Keep only those countries that have exactly one distinct neighbouring country.

```

HAVING (COUNT(DISTINCT ?neighbour) = 1)
}
}

```

Output:

```

lonelyCountryCount
19

```

Task 2: Academic Lineage

Task: Count doctoral students advised by someone who was themselves advised by Alan Turing.

I interpreted this task as a two-step academic lineage problem. The query needed to follow the doctoral advisor relationship twice, first from a student to their advisor, and then from that advisor to Alan Turing. I therefore used a property-path style solution rather than writing the same relationship separately multiple times. This made the query more direct and better matched the hint in the brief.

SPARQL Query:

```
# Task 2: Academic Lineage
```

```
# Objective: Count the number of doctoral students whose doctoral advisor was themselves
advised by Alan Turing.
```

```
# The query returns one final count of distinct students in this two-step academic lineage.
```

```
SELECT (COUNT(DISTINCT ?student) AS ?academicLineageCount)
```

```
WHERE {
```

```
  # P184 = doctoral advisor
```

```
  # Q7251 = Alan Turing
```

```
  # Using the property path twice means:
```

```
  # ?student -> doctoral advisor -> doctoral advisor -> Alan Turing
```

```
  # In other words, the student's advisor was advised by Alan Turing.
```

```
?student wdt:P184/wdt:P184 wd:Q7251 .
}
```

Output:

```
academicLineageCount
27
```

Task 3: The “EGOT” Gap

Task: Count humans who have won an Academy Award but never a Golden Globe Award.

This task required negation logic. I first restricted results to humans and then selected those who had received an Academy Award. After that, I excluded any person who had also received a Golden Globe Award. I counted distinct people in the final step to avoid duplicates.

SPARQL Query:

```
# Task 3: The "EGOT" Gap
```

```
# Objective: Count the number of humans who have won an Academy Award but have never won
any Golden Globe Award.
```

```
SELECT (COUNT(DISTINCT ?person) AS ?egotGapCount)
```

```
WHERE {
```

```
  # Restrict results to humans
```

```
  # P31 = instance of
```

```
  # Q5 = human
```

```
  ?person wdt:P31 wd:Q5 .
```

```
  # Identify people who have received an Academy Award
```

```
  # P166 = award received
```

```
  # Q19020 = Academy Awards
```

```
  ?person wdt:P166 wd:Q19020 .
```

```

# Exclude anyone who has received any Golden Globe award.

# Instead of matching a single award entity, we match all subclasses of the Golden Globe
Awards category.

FILTER NOT EXISTS {
    ?person wdt:P166 ?ggAward .
    ?ggAward wdt:P279* wd:Q1011547 .
}
}

```

Output:

```

egotGapCount
8

```

Task 4: Scientific Gender Gap

Task: Show the count of female and male Nobel Prize winners in Physics.

In this task, the output requirement mattered because the brief asks for counts for both genders. I therefore grouped the results by gender. I restricted results to humans who had received the Nobel Prize in Physics and retrieved their recorded sex or gender values.

SPARQL Query:

```

# Task 4: Scientific Gender Gap

# Objective: Count the number of female and male Nobel Prize in Physics winners.

SELECT ?genderLabel (COUNT(DISTINCT ?person) AS ?winnerCount)
WHERE {

    # Restrict entities to humans

```

```

# P31 = instance of
# Q5 = human
?person wdt:P31 wd:Q5 .

# Identify people who received the Nobel Prize in Physics
# P166 = award received
# Q38104 = Nobel Prize in Physics
?person wdt:P166 wd:Q38104 .

# Retrieve the recorded gender of the person
# P21 = sex or gender
?person wdt:P21 ?gender .

# Limit results to female and male categories
VALUES ?gender { wd:Q6581072 wd:Q6581097 }

# Retrieve readable labels for output
SERVICE wikibase:label { bd:serviceParam wikibase:language "en". }
}

# Group results so that the count is computed separately for each gender
GROUP BY ?genderLabel

# Order output alphabetically by gender label for readability
ORDER BY ?genderLabel

```

Output:

genderLabel	winnerCount
female	5
male	224

Task 5: Polyglot Cinema

Task: Count films that have original audio tracks in more than 3 different languages.

I treated this as a grouped counting query. I first restricted results to films and retrieved the original language values associated with each film. I grouped by film and counted distinct languages, keeping only films with more than three languages.

SPARQL Query:

Task 5: Polyglot Cinema

Objective: Count the number of films whose original audio track exists in more than three different languages.

SELECT (COUNT(*) AS ?polyglotFilmCount)

WHERE {

Subquery identifies films that have more than three distinct original languages.

{

SELECT ?film

WHERE {

Restrict entities to films

P31 = instance of

Q11424 = film

?film wdt:P31 wd:Q11424 .

Retrieve the original language(s) of each film

P364 = original language of film or TV show

?film wdt:P364 ?language .

}

Group results by film so we can count the number of languages per film

GROUP BY ?film

```
# Keep only films that have more than three distinct languages
HAVING (COUNT(DISTINCT ?language) > 3)
}
}
```

Output:

```
polyglotFilmCount
691
```

Task 6: Political Education

Task: Count entities who held head-of-government positions and also hold doctorate-level degrees.

This task required combining two conditions across the same entity: a political role and an academic qualification. I expanded subclasses for both “head of government” and “doctorate” so that related positions and degrees were included. I then counted distinct people satisfying both conditions.

SPARQL Query:

```
# Task 6: Political Education
```

```
# Objective: Count the number of entities who held the position of head of government (or a
subclass of it) and also hold a doctorate degree (or a subclass of it).
```

```
SELECT (COUNT(DISTINCT ?person) AS ?politicalEducationCount)
```

```
WHERE {
```

```
  # First collect all positions that are equal to, or subclasses of, head of government. Using a
  subquery helps reduce the search space before matching people.
```

```
{
```



```

SELECT DISTINCT ?hogPosition
WHERE {
  # P279 = subclass of
  # Q2285706 = head of government
  ?hogPosition wdt:P279* wd:Q2285706 .
}
}

# Then collect all academic degrees that are equal to, or subclasses of, doctorate. This follows
the task hint to capture doctorate subclasses.
{
  SELECT DISTINCT ?doctorateDegree
  WHERE {
    # P279 = subclass of
    # Q849697 = doctorate
    ?doctorateDegree wdt:P279* wd:Q849697 .
  }
}

# Match people who held one of the qualifying head-of-government positions and also hold one
of the qualifying doctorate-level degrees.
# P39 = position held
# P512 = academic degree
?person wdt:P39 ?hogPosition .
?person wdt:P512 ?doctorateDegree .
}

```

Output:

politicalEducationCount

632

Task 7: The “27 Club” & Others

Task: Count musical groups that had more than 2 members who died before 2000.

The key modelling point was that death dates belong to members rather than the group itself. I therefore linked group members to their musical groups, retrieved member death dates, filtered for deaths before 2000, grouped by group, and kept groups where more than two members met the condition.

SPARQL Query:

Task 7: The “27 Club” & Others

Objective: Count the number of musical groups that have had more than two members who died before the year 2000.

```
SELECT (COUNT(*) AS ?groupCount)
```

```
WHERE {
```

```
  # Subquery identifies musical groups that have more than two members with a recorded death
  # date before 2000.
```

```
{
```

```
  SELECT ?group
```

```
  WHERE {
```

```
    # Restrict entities to musical groups
```

```
    # P31 = instance of
```

```
    # Q215380 = musical group
```

```
    ?group wdt:P31 wd:Q215380 .
```

```
    # Use the member's relationship to the group, not the group itself.
```

```
    # P463 = member of
```

```
    ?member wdt:P463 ?group .
```

```
    # Retrieve the death date of each member
```

```
    # P570 = date of death
```

```
    ?member wdt:P570 ?deathDate .
```

```
    # Keep only members who died before 2000
```

```
    FILTER(YEAR(?deathDate) < 2000)
  }
  # Group by musical group so that we can count qualifying members
  GROUP BY ?group
  # Keep only groups with more than two distinct deceased members
  HAVING (COUNT(DISTINCT ?member) > 2)
}
}
```

Output:

```
groupCount
59
```

Task 8: Chemical Composition

Task: Count chemical compounds that contain oxygen but do not contain any carbon atoms.

To address this task, I first examined how chemical composition is represented in Wikidata. Although elemental composition could ideally be queried using structural relationships such as has part (P527), I observed that this property is not consistently used for representing elements within compounds. Because of this limitation, I instead used the chemical formula property (P274), which is widely available for chemical compounds.

Chemical formulas follow standard notation where elements appear as symbols (for example H₂O, NaCl, or CO₂). I therefore analysed the formula string using regular expressions. The query first filters formulas containing O, indicating the presence of oxygen. It then excludes compounds containing carbon by detecting the element symbol C. An additional filtering step was required because several other element symbols begin with the letter C (such as Co, Cr, Cu, Ca, and Cl). To avoid incorrectly excluding these compounds, the regular expression only matches C when it appears as a complete element symbol within the formula. Using this approach, the query counts compounds whose formulas indicate the presence of oxygen but not carbon.

SPARQL Query:

Task 8: Chemical Composition

Objective: Count chemical compounds that contain Oxygen but do not contain Carbon.

```
SELECT (COUNT(DISTINCT ?compound) AS ?oxygenWithoutCarbonCount)
```

```
WHERE {
```

```
  # Restrict results to chemical compounds
```

```
  # P31 = instance of
```

```
  # Q11173 = chemical compound
```

```
  ?compound wdt:P31 wd:Q11173 .
```

```
  # Retrieve the chemical formula
```

```
  # P274 = chemical formula
```

```
  ?compound wdt:P274 ?formula .
```

```
  # Keep compounds whose formula contains Oxygen
```

```
  FILTER(REGEX(?formula, "O"))
```

```
  # Exclude formulas containing Carbon as an element symbol
```

```
  FILTER(!REGEX(?formula, "C([0-9]|[A-Z]|$)"))
```

```
}
```

Output:

```
oxygenWithoutCarbonCount
```

```
9
```

Task 9: Data Consistency Audit

Task: Count humans that have a recorded date of death but are missing a date of birth.

For this task, I followed the hint in the brief and approached the problem as a missing-data query using FILTER. I first restricted the result set to humans and then required that each person must have a recorded date of death. After that, I used an OPTIONAL pattern to attempt to retrieve the date of birth, and then applied a filter to keep only those cases where no birth date was bound. In other words, I treated the absence of a direct birth-date statement as the condition for “missing Date of Birth.”

Before trying to compute the full aggregate count, I verified the logic with a preview query. The preview returned valid human entities that had a date of death but no recorded date of birth, which confirmed that the logical structure of the query was correct.

When I then attempted the full aggregate count on the public Wikidata Query Service, the endpoint timed out repeatedly. Since the preview query successfully returned valid matching entities, the repeated timeout during the aggregate query most likely reflects a performance limitation of the public WDQS endpoint rather than an issue with the query logic. I have therefore kept the FILTER-based aggregate query as my final solution because it aligns best with the task hint and correctly expresses the intended data-consistency condition. The brief explicitly states that correct SPARQL logic should still receive credit even if live Wikidata changes or service behaviour affects the exact result.

Preview query used to verify the logic: (Not added in the submission code)

```
SELECT ?person
WHERE {
  ?person wdt:P31 wd:Q5 .
  ?person wdt:P570 ?deathDate .
  OPTIONAL { ?person wdt:P569 ?birthDate . }
  FILTER(!BOUND(?birthDate))
}
LIMIT 10
```

SPARQL Query:

Task 9: Data Consistency Audit

Objective: Count the number of humans that have a recorded date of death but are missing a recorded date of birth.

```
SELECT (COUNT(*) AS ?missingBirthCount)
```

```
WHERE {
```

```
  # First identify humans with a death date, then use FILTER to keep only those where no birth
  date is present.
```

```
{
```

```
  SELECT DISTINCT ?person
```

```
  WHERE {
```

```
    # P31 = instance of
```

```
    # Q5 = human
```

```
    ?person wdt:P31 wd:Q5 .
```

```
    # P570 = date of death
```

```
    ?person wdt:P570 ?deathDate .
```

```
    # P569 = date of birth
```

```
    OPTIONAL { ?person wdt:P569 ?birthDate . }
```

```
    # Keep only people for whom no birth date is recorded
```

```
    FILTER(!BOUND(?birthDate))
```

```
  }
```

```
}
```

```
}
```

Output:

```
missingBirthCount
```

```
WDQS timeout on live aggregate.
```

**FILTER based preview query verified matching humans*

Task 10: The “Modern Metropolis”

Task: Count cities with population greater than 1,000,000 located in countries founded after 1950.

This task required filtering across two connected entities. I first filtered cities by population and then followed the city–country relationship to retrieve the founding date of each country. I then filtered countries founded after 1950 and counted distinct cities satisfying both conditions.

SPARQL Query:

Task 10: The “Modern Metropolis”

Objective: Count the number of cities with a population greater than 1,000,000 that are located in a country founded after the year 1950.

SELECT (COUNT(DISTINCT ?city) AS ?modernMetropolisCount)

WHERE {

Restrict results to cities

P31 = instance of

Q515 = city

?city wdt:P31 wd:Q515 .

Retrieve the population of the city

P1082 = population

?city wdt:P1082 ?population .

Connect the city to its country

P17 = country

?city wdt:P17 ?country .

Retrieve the founding / inception date of the country

P571 = inception

?country wdt:P571 ?founded .

```
# Keep only cities with population greater than 1,000,000
FILTER(?population > 1000000)

# Keep only countries founded after 1950
FILTER(YEAR(?founded) > 1950)
}
```

Output:

```
modernMetropolisCount
64
```

Conclusion

Through these ten tasks, I used SPARQL to retrieve and analyse structured facts from the Wikidata knowledge graph. The exercises required translating natural language questions into graph queries that operate over entities, properties, and relationships rather than traditional tabular structures. Across the tasks, I applied several key SPARQL techniques, including aggregation with GROUP BY and HAVING, property path traversal for multi-step relationships, negation patterns for exclusion queries, and FILTER based approaches for detecting missing data.

Working with a live knowledge graph also highlighted the practical challenges of querying real datasets. In particular, Task 8 demonstrated how the way information is modelled in Wikidata can influence the query strategy that needs to be adopted. While elemental composition could ideally be queried through structural relationships, I observed that these relationships were not consistently populated and therefore used the chemical formula property instead, analysing it with regular expressions to identify the required elements. Task 9, in contrast, illustrated how performance constraints of the public query endpoint can affect complex queries even when the underlying logic is correct. Rather than treating these situations as query errors, I verified the query logic using preview queries and explained the limitations encountered.

Overall, completing these tasks provided practical experience in reasoning about graph-structured data and expressing that reasoning using SPARQL, and expressing that reasoning using SPARQL. By combining entity filtering, relationship traversal, aggregation, and missing-data detection, these queries demonstrate how structured knowledge can be systematically retrieved and analysed within a large collaborative knowledge graph such as Wikidata.